The NetCDF Developer's Handbook — Chapter Reference

Edward Hartnett | 16 Chapters | 2026 Edition

Ch. 1 — Introduction

- Self-describing & portable: metadata travels with the file.
- NC4 adds chunking, compression, parallel I/O for TB-scale data.
- NASA, NOAA, ECMWF, CMIP all standardized on netCDF; hundreds of PB.
- Workflow: create → define dims/vars → write → close.

Ch. 2 — Obtaining NetCDF

- One install method; mixing pkg-mgr + source causes mismatches.
- Linux: apt/dnf; macOS: Homebrew; HPC: Conda or Spack.
- Build order: zlib → HDF5 → netcdf-c → netcdf-fortran.
- Parallel: build HDF5 with `-enable-parallel` via `mpicc`.
- Java reads are pure Java; NC4 writes require JNI (`cdm-core`).
- Verify: `nc-config -all` + smoke test linking `-lnetcdf`.

Ch. 3 — Data Models

- Classic: dims+vars+attrs, one unlimited dim; maximum portability.
- Enhanced (NC4): groups, multiple unlimited dims, user-defined types.
- Coordinate variable: 1-D var with same name as its dimension.
- User types: compound, VLEN, enum, opaque; Fortran support varies.
- Use classic for compatibility; enhanced for complex/hierarchical data.

Ch. 4 — Binary Formats

- Five formats: Classic, 64-bit Offset, CDF-5, NC4/HDF5, ncZarr.
- Format is transparent on read; only matters at file creation.
- NC4/HDF5: recommended default—compression, groups, parallel I/O.
- CDF-5: classic model, no size limits; best for PnetCDF parallel I/O.
- ncZarr: cloud object storage (S3/GCS/Azure); chunk-per-object layout.
- `nccopy -k` converts formats; enhanced→classic fails if groups exist.

Ch. 5 — Command-Line Utilities

- `ncdump -h`: structure; `-hs`: adds chunk/compression info.
- `ncgen`: CDL text → binary; `-lc/-lf` generates C/Fortran code.
- ncdump→edit→ncgen round-trip for metadata fixes.
- `nccopy -d1 -s`: compress; `-c`: rechunk for access pattern.
- NCO: `ncrcat` (concat), `ncks` (subset), `ncap2` (math).

Ch. 6 — C Examples

- Error idiom: `if ((ret=nc_xxx(...))) ERR(ret);`
- Progression: simple_2D → coord → compression → types → groups → parallel.
- Parallel I/O: `nc_create_par()` with MPI communicator.
- CF attributes (`units`, `standard_name`) added in coord example.

Ch. 7 — Fortran Examples

- F77: `nf_*`; F90: `nf90_*` with optional keyword args (preferred).
- Array dim order reversed vs. C (column-major Fortran).
- Compile: `gfortran prog.f90 -lnetcdff -lnetcdf`
- Same progression as C; compound/VLEN support limited in Fortran.

Ch. 8 — Java (NetCDF-Java)

- Pure Java re-implementation (CDM); not a C wrapper.
- Reads NC-3/4, HDF5, GRIB; NC4 writes require JNI.
- Add `edu.ucar:cdm-core` or `netcdfAll` via Maven/Gradle.
- NcML: XML metadata modification and multi-file aggregation.
- Not thread-safe: one `NetcdfFile` per thread.

Ch. 9 — Attributes & Conventions

- CF: `standard_name`, `units` (UDUNITS), `coordinates`, `cell_methods`.
- ACDD: discovery metadata (`title`, `institution`, `history`).
- `_FillValue` must match variable type; set before writing data.
- Time: `"days since 1970-01-01"` + `calendar` attribute.
- Validate compliance with cf-checker or IOOS Compliance Checker.

Ch. 10 — Performance Fundamentals

- Chunk shape drives both I/O speed and compression effectiveness.
- Match chunks to dominant access pattern (time-slice vs. point).
- Cache: tune with `nc_set_chunk_cache(size, nslots, preemption)`.
- Shuffle + Zstandard level 1 is best typical combination.
- No-fill mode avoids unnecessary pre-fill write overhead.

Ch. 11 — Parallel I/O

- `nc_create_par`/`nc_open_par` with MPI communicator.
- Collective I/O (`NC_COLLECTIVE`) is usually faster than independent.
- Align chunk shape with MPI domain decomposition layout.
- PnetCDF: parallel classic/CDF-5 without HDF5.
- PIO library (NCAR): async I/O for coupled model systems on HPC.

Ch. 12 — Compression

- Always apply shuffle filter before any lossless compressor.
- Zstandard ($\geq$ NC 4.9): best ratio+speed; use level 1.
- LZ4 (NEP): fastest decode; good for interactive access.
- BZIP2: highest ratio, slowest; rarely justified.
- Quantization (BitGroom/GranularBitRound): lossy pre-compression.
- HDF5 filter plugins: runtime loadable via `nc_def_var_filter()`.

Ch. 13 — Architecture & Extensibility

- Dispatch table routes `nc_*` calls to format-specific drivers.
- Add custom format via `nc_def_user_format()`.
- NEP: LZ4/BZIP2 compression, CDF, GeoTIFF format readers.
- Avoid V2 API (`ncvgt` etc.) in new code.

Ch. 14 — OPeNDAP Remote Access

- `nc_open(URL, NC_NOWRITE, &id)`—transparent remote read.
- Constraint expressions subset server-side (saves bandwidth).
- DAP2 and DAP4 both handled automatically by netCDF-C.
- Auth: use `.dodsrc`/`.netrc`; explore with `ncdump -h URL`.
- Batch multiple vars into one request; request only needed slices.

Ch. 15 — Performance Benchmarking

- `bm_file`: built-in benchmark; varies chunk, compression, pattern.
- Time I/O: `clock_gettime()` (C) or `cpu_time()` (Fortran).
- `h5stat`/`ncdump -hs`: inspect actual chunk layout.
- Shell loop + `nccopy`: sweep chunk-size/compression combos.
- Compare write speed, read speed, and file size per configuration.

Ch. 16 — Past, Present & Future

- 1989 Classic; 2008 NC4/HDF5; 2020+ ncZarr, Zstandard, NEP.
- Current: netcdf-c 4.10, netcdf-fortran 4.6.3, Java 5.x, NCO, NEP.
- Maintained by Unidata, Northwestern, NCAR, UC Irvine, OPeNDAP.
- Every release maintains backward compatibility with prior formats.
- Future: cloud-native I/O, richer compression, community-driven growth.